

# TTP – Time-Triggered Protocol for Fault-Tolerant Real-Time Systems

Milestone 2: Scientific Contextualization and Tradeoff Analysis

Jaleel Ur Rehman  
Electronics Engineering  
Hochschule Hamm-Lippstadt  
jaleel.ur-rehman@stud.hshl.de

## Positioning TTP in the Protocol Landscape

---

TTP sits at the extreme time-triggered end of the real-time protocol design space. The central design dimension is *how bandwidth is allocated*: statically (time-triggered, deterministic) vs. dynamically (event-triggered, reactive). Table 1 maps the major protocols along this axis.

Table 1: Real-Time Communication Protocols – Design Space Overview

| Protocol   | Scheduling      | Fault Tolerance  | Primary Domain              |
|------------|-----------------|------------------|-----------------------------|
| CAN        | Event (CSMA/CA) | None native      | Automotive body electronics |
| J1850      | Token           | Partial          | Automotive diagnostics      |
| TTP        | Static TDMA     | Fully integrated | Safety-critical control     |
| TTCAN      | Hybrid          | Partial          | CAN upgrade path            |
| FlexRay    | Hybrid (TT+ET)  | Partial          | Automotive chassis          |
| ARINC 429  | Time-triggered  | None             | Avionics (simplex)          |
| TSN (IEEE) | Time-aware ETH  | None native      | Industrial / automotive ETH |

## Competing and Neighboring Approaches

---

### CAN (Controller Area Network)

- Arbitration via CSMA/CA: highest-priority message wins bus access; lower-priority messages are delayed.
- Worst-case latency is **load-dependent and practically unbounded** at high bus utilization.

- No integrated clock synchronization, no membership service, no replica determinism.
- Data efficiency: 25.3% (vs. TTP's 53.3%) – every frame carries sender address and message ID.
- **Suitable:** body electronics, diagnostics, low-load non-safety-critical applications.
- **Unsuitable:** hard real-time safety functions requiring bounded worst-case latency.

### J1850 (Token-Based)

- Token passing provides partial predictability: worst-case latency is bounded by the token rotation time.
- Explicit token frames add 160 bits of overhead per loop; data efficiency 27.1%.
- Token loss requires a recovery protocol – introduces non-deterministic latency spikes.
- Not designed for fault-tolerant clock synchronization or integrated membership.
- Intermediate position between CAN and TTP: more predictable than CAN, less than TTP.

### TTTCAN (Time-Triggered CAN)

- Adds a time-triggered master and a TDMA scheduling layer on top of standard CAN hardware.
- Inherits CAN's 1 Mbit/s bandwidth ceiling and the fail-silent property is *not* enforced at the hardware level.
- Clock synchronization and membership are not as tightly integrated as in TTP.
- Useful as a migration path for existing CAN infrastructure; not a clean-sheet fault-tolerant design.

### FlexRay

- **Hybrid architecture:** static segment (time-triggered, like TTP) + dynamic segment (event-triggered, like CAN).
- Up to 10 Mbit/s per channel; dual-channel redundancy.
- Does not enforce fail-silence at the protocol level; membership and fault tolerance are less rigorous than TTP.
- The dynamic segment accommodates aperiodic/bursty traffic – addressing TTP's main inflexibility.
- Became the dominant automotive chassis bus in the 2000s because it supports both periodic control (static segment) and aperiodic diagnostics (dynamic segment).
- FlexRay's design was directly influenced by TTP concepts.

## ARINC 429 (Avionics)

- Unidirectional time-triggered broadcast: each transmitter has dedicated physical wires to each receiver.
- Simple, well-certified, highly predictable.
- No integrated membership, no fault-tolerant clock synchronization, no C-state concept.
- Topology is simplex (one-to-many); no shared bus – eliminates bus contention but increases wiring weight.
- TTP is architecturally more capable; ARINC 429 is simpler and carries decades of aviation certification history.

## Time-Sensitive Networking – TSN (IEEE 802.1Qbv / 802.1AS)

- Adds time-aware traffic shaping (scheduled gates) and time synchronization (gPTP) to standard Ethernet.
- Shares TTP’s core insight: time-triggered transmission eliminates contention and jitter.
- Does *not* provide integrated membership, fail-silent enforcement, or C-state agreement – fault tolerance must be added by higher-layer protocols.
- Scales to 1 Gbit/s and beyond; supports switched, heterogeneous topologies.
- Modern automotive (OPEN Alliance, AUTOSAR) and industrial (OPC-UA over TSN) stacks are converging on TSN as the future basis.

## Data Efficiency Comparison

---

Table 2: Message Loop Size: 8 Nodes, 16-Message Loop, 2 Bytes/Message [1]

|                          | Token | J1850 | CAN   | TTP          |
|--------------------------|-------|-------|-------|--------------|
| Message overhead (bits)  | 656   | 688   | 752   | 224          |
| Token overhead (bits)    | 160   | –     | –     | –            |
| Message data (bits)      | 256   | 256   | 256   | 256          |
| <b>Total loop (bits)</b> | 1072  | 944   | 1008  | <b>480</b>   |
| <b>Data efficiency</b>   | 24.0% | 27.1% | 25.3% | <b>53.3%</b> |

TTP’s 53.3% efficiency is a *structural* advantage: it follows directly from eliminating sender addresses, message names, and ACK frames through a-priori schedule knowledge and C-state agreement. Dynamic protocols cannot match this without losing their adaptive capability.

## Key Engineering Tradeoffs

---

- **Determinism vs. flexibility:**

- TTP: full TDMA determinism, load-independent worst-case timing, trivial schedulability analysis.
  - Cost: any topology or message change requires schedule redeployment to all nodes. No hot-plug.
  - CAN/FlexRay dynamic segment: flexible, but worst-case timing is load-dependent and harder to certify.
- **Data efficiency vs. adaptability:**
    - TTP achieves 53.3% only because message content and timing are known a priori.
    - Dynamic protocols must carry identity information per frame, paying 24–27% efficiency in exchange for runtime flexibility.
- **Integrated fault tolerance vs. protocol simplicity:**
    - TTP bundles clock sync, membership, and redundancy in one protocol – reduces inter-component integration bugs and simplifies safety cases.
    - CAN keeps the protocol simple and pushes fault tolerance to higher layers or separate hardware – more flexibility but more integration risk.
- **Fail-silence assumption vs. Byzantine resilience:**
    - Fail-silence is cheaper than Byzantine fault tolerance (BFT) but weaker.
    - BFT protocols require  $3f + 1$  nodes to tolerate  $f$  Byzantine faults
    - TTP explicitly excludes Byzantine faults from its failure model – a significant gap for applications where hardware fault modes include inconsistent outputs to multiple receivers.
- **Static bandwidth allocation vs. utilization under low load:**
    - TTP wastes allocated TDMA slots when nodes have no data to send.
    - Event-triggered protocols use bandwidth on demand, achieving better utilization under sparse or bursty traffic.
    - Relevant for mixed-function ECUs that combine periodic safety-critical loops with aperiodic diagnostic traffic.
- **Predictable worst-case vs. good average-case:**
    - TTP: tight, load-independent upper bound on message latency. Ideal for hard real-time control loops.
    - CAN: very low average latency at low load, but no firm upper bound. Acceptable for best-effort or soft real-time functions.

## Why Related Work Is Sparse in Some Areas

---

- TTP is primarily a single-group contribution (Kopetz, TU Vienna). The design space for *purely* static TDMA with fully integrated fault tolerance is narrow – subsequent industrial work (FlexRay, TSN) moved toward hybrid or Ethernet-based architectures.
- Byzantine fault tolerance in real-time bus protocols remains an open research area. TTP explicitly excludes it and no widely deployed successor has fully solved it at the bus protocol level.
- Mixed-criticality scheduling is a parallel line of work that addresses TTP’s limitation of treating all tasks with equal criticality, but in a software scheduling context rather than a bus protocol context.
- Dynamic TDMA reconfiguration (hot-plug, online schedule updates) is an area where TTP has no solution; this is an active research gap in real-time communication.

## References

---

- [1] H. Kopetz and G. Grünsteidl, “TTP – A Time-Triggered Protocol for Fault-Tolerant Real-Time Systems,” in *Proc. 23rd Int. Symp. Fault-Tolerant Computing (FTCS-23)*, pp. 524–533, Toulouse, France, June 1993.
- [2] H. Kopetz, “Event-Triggered versus Time-Triggered Real-Time Systems,” in *Proc. Int. Workshop on Operating Systems of the 90s and Beyond*, LNCS vol. 563, pp. 87–101, Springer-Verlag, 1991.
- [3] SAE Handbook, Vol. 2. Society of Automotive Engineers, PA, USA, 1992.
- [4] FlexRay Consortium, *FlexRay Communications System – Protocol Specification*, Version 2.1, 2005.
- [5] IEEE Standards Association, “IEEE 802.1Qbv – Enhancements for Scheduled Traffic,” IEEE Standard, 2015.
- [6] L. Lamport, R. Shostak, and M. Pease, “The Byzantine Generals Problem,” *ACM Trans. on Programming Languages and Systems*, vol. 4, no. 3, pp. 382–401, 1982.
- [7] S. Vestal, “Preemptive Scheduling of Multi-Criticality Systems with Varying Degrees of Execution Time Assurance,” in *Proc. 28th IEEE Real-Time Systems Symp. (RTSS)*, pp. 239–243, Tucson, AZ, USA, 2007.
- [8] H. Kopetz and W. Ochsenreiter, “Clock Synchronization in Distributed Real-Time Systems,” *IEEE Trans. Computers*, vol. 36, no. 8, pp. 933–940, Aug. 1987.